MANBA KOD (SOURCE CODE)

"Tilora" veb-platformasining foydalanuvchi profili va boshqaruv dasturiy ta'minoti

EHM uchun dasturni rasmiy ro'yxatdan o'tkazish (DGU) arizasiga ilova
Ilova sanasi: 15.10.2026

Tarkib (eng muhim modullar tanlab olingan, listing qisqartirilgan):

backend/app/main.py FastAPI ilovasi: routerlar, middleware, xavfsizlik sarlavhalari	332 qator
backend/app/config.py Sozlamalar: JWT kaliti, baza, xavfsizlik parametrlari	130 qator
backend/app/database.py SQLAlchemy ulanishi va sessiya	21 qator
backend/app/core/security.py Parol xeshlash (PBKDF2-SHA256) va JWT (HMAC-SHA256)	114 qator
backend/app/core/deps.py Avtorizatsiya bog'liqliklari: joriy foydalanuvchi, rollar	164 qator
backend/app/core/access.py Tarif (obuna) bo'yicha kirish tekshiruvi	15 qator
backend/app/core/ratelimit.py Kirish urinishlarini cheklash (rate limiting)	73 qator
backend/app/core/errors.py Xatolarni yagona formatda qaytarish	50 qator
backend/app/models/entities.py Ma'lumotlar bazasi modellari (User, Group, AuditLog va b.)	503 qator
backend/app/api/auth.py Auth router: ro'yxatdan o'tish, kirish, profil, parolni o'zgartirish	469 qator
backend/app/api/admin.py Admin router: foydalanuvchilarni tasdiqlash, tarif, parolni tiklash	393 qator

Jami: 11 ta fayl, 2264 qator Python kodi.

Texnik muhit: Python 3.11, FastAPI, SQLAlchemy 2.0, Pydantic 2, PostgreSQL 16 / SQLite.
Eslatma: maxfiy qiymatlar (kalitlar, tuzlar, standart parollar) listingda <REDACTED> belgisi bilan almashtirilgan.
Mijoz qismi (JavaScript/HTML5/CSS3) va qolgan server modullari hajm cheklovi sababli listingga kiritilmagan.

Fayl: backend/app/main.py

FastAPI ilovasi: routerlar, middleware, xavfsizlik sarlavhalari

```
import os
import sys
import ipaddress

# Windows Application Control ujson DLL bloklanishidan himoya
sys.modules['ujson'] = None

# HTTPS: tizim (Windows/macOS) sertifikat omboridan foydalanish.
# Antivirus (Avast va h.k.) yoki korporativ proksi HTTPS ni o'z sertifikati bilan
# "tekshirsa", Python'ning o'z CA ro'yxati uni tanimaydi va AI API ulanmaydi.
try:
    import truststore
    truststore.inject_into_ssl()
except Exception:
    pass

from fastapi import FastAPI, Request
from fastapi.middleware.cors import CORSMiddleware
from fastapi.staticfiles import StaticFiles
from fastapi.responses import FileResponse, JSONResponse

from app.config import settings
from app.database import engine, Base
import app.models

from app.api.auth import router as auth_router
from app.api.student import router as student_router
from app.api.lesson import router as lesson_router
from app.api.ai import router as ai_router
from app.api.teacher import router as teacher_router
from app.api.teacher_diagnostic import router as teacher_diag_router
from app.api.feedback import router as feedback_router
from app.api.research import router as research_router
from app.api.admin import router as admin_router
from app.api.proposals import router as proposals_router
from app.api.api_keys import router as api_keys_router
from app.api.content import router as content_router
from app.api.teacher_panel import router as teacher_panel_router

# Ma'lumotlar bazasini yaratish
Base.metadata.create_all(bind=engine)

def _ensure_columns():
    """create_all mavjud jadvalga ustun qo'shmaydi; yetishmayotgan ustunlarni ALTER TABLE bilan qo'shamiz."""
    from sqlalchemy import inspect, text
    insp = inspect(engine)
    needed = {
        "custom_diagnostic_questions": [("subject_id", "VARCHAR(36)")],
        "custom_survey_questions": [("subject_id", "VARCHAR(36)")],
        "teacher_proposals": [
            ("sender_id", "VARCHAR(36)"),
            ("receiver_id", "VARCHAR(36)"),
            ("created_at", "DATETIME")
        ]
    }
    with engine.begin() as conn:
        for table, cols in needed.items():
            if table not in insp.get_table_names():
                continue
            existing = {c["name"] for c in insp.get_columns(table)}
            for col, ctype in cols:
                if col not in existing:
                    conn.execute(text(f"ALTER TABLE {table} ADD COLUMN {col} {ctype}"))

_ensure_columns()

tags_metadata = [
    {
        "name": "Autentifikatsiya",
        "description": "Foydalanuvchi tizimga kirishi (login), ro'yxatdan o'tishi (register), JWT token olish va obuna tekshiruvi."
    },
    {
        "name": "Talaba",
        "description": "Talabaning 3 o'lchovli profili (CEFR ko'nikmalar, ESP kasbiy yo'nalish, FLCAS kognitiv tashvish), 16 haftalik yo'l va o'quv materiallari."
    },
    {
        "name": "Mashg'ulot",
        "description": "5 bosqichli adaptiv mashg'ulot modeli: 1. Dars maqsadi, 2. Skaffolding ko'mak, 3. Mustaqil nutqiy mahsulot (AI yopiq - 1-tamoyil), 4. Sababli tahlil (2-tamoyil), 5. Metakognitiv refleksiya (4-tamoyil)."
    },
    {
        "name": "Takliflar va Pedagogik Muloqot",
        "description": "3-tamoyil: Pedagogik suverenitet. O'qituvchilar, talabalar va AI o'rtasidagi takliflar almashinuvi, tasdiqlash, o'zgartirish va rad etish boshqaruvi."
    },
    {
```

```
        "name": "O'qituvchi",
        "description": "Fanlar boshqaruvi, AI orqali 16 haftalik sillabus generatsiyasi, testlar, taqdimotlar va dars rejalari yaratish."
    },
    {
        "name": "O'qituvchi: Diagnostika banki",
        "description": "CEFR 4 ko'nikma bo'yicha diagnostik test savollari, audio/TTS yozuvlar va FLCAS anketa savollari banki boshqaruvi."
    },
    {
        "name": "Sun'iy Intellekt Vositalari",
        "description": "DeepSeek v4 va TAAM AI lingvistik tahlil mexanizmi, skaffolding tavsiyalari va matn baholash."
    },
    {
        "name": "Izohlar",
        "description": "Hattie & Timperley modeli asosidagi sababli pedagogik izohlar va ona tili (L1) interferensiyasi tahlili."
    },
    {
        "name": "Ma'lumotnoma (Kontent)",
        "description": "Ochiq ma'lumotnoma: ELT glossariysi, CEFR daraja tavsiflari, L1 interferensiya naqshlari, metodik asoslar va FAQ."
    },
    {
        "name": "Tadqiqotchi",
        "description": "DSc Dissertatsiyasi 4-bobi uchun matematik-statistik hisob-kitoblar, 4.2–4.9 jadvallar, Bespalko K ko'rsatkichi, Word (.docx) va Excel (.xlsx) eksporti."
    },
    {
        "name": "Administrator",
        "description": "Super administrator paneli: o'qituvchi va talabalarni tasdiqlash, guruhlar va tizim monitoringi."
    },
    {
        "name": "API Kalitlari (API Keys)",
        "description": "Dasturchi (developer) va Super Admin (admin) uchun maxfiy API kalitlarini yaratish, bekor qilish va boshqarish."
    }
]

app_description = """
#   TILORA Platformasi RESTful API Hujjatlari

Mazkur API pedagogika fanlari doktori (DSc) ilmiy tadqiqoti doirasida yaratilgan:
- **Tadqiqotchi:** Ikramova Aziza Aminovna
- **Ixtisoslik:** 13.00.02 – Ta'lim va tarbiya nazariyasi va metodikasi
- **Mavzu:** «Ingliz tilini sun'iy intellekt texnologiyalari asosida o'qitish metodikasini takomillashtirish»
- **Muassasa:** Buxoro davlat universiteti, Chet tillar fakulteti

---

###    To'rt Asosiy Pedagogik Tamoyil:
1. **1-Qoida: Mustaqil bosqichda AI yopiq** (`POST /api/lesson/{id}/production`) — talaba matn yaratayotganida tashqi AI qulflanadi (server tomonda, foydalanuvchi holati bo'yicha).
2. **2-Qoida: Sabab shablonda ochiq** (`GET /api/lesson/{id}/feedback`) — Hattie & Timperley modeli bo'yicha xato sababi ko'rsatiladi.
3. **3-Qoida: Yakuniy qaror o'qituvchida** (`POST /api/proposals/{id}/approve`) — Pedagogik suverenitet saqlanadi.
4. **4-Qoida: Xulosani talaba chiqaradi** (`POST /api/lesson/{id}/reflection`) — Talaba o'z bahosi va AI bahosi tafovutini tahlil qiladi.

---

###   Autentifikatsiya:
Barcha himoyalangan endpointlar uchun HTTP sarlavhasi (Header) talab etiladi:
`Authorization: Bearer <access_token>`
Token olish uchun: `POST /api/auth/login`
"""

app = FastAPI(
    title=settings.PROJECT_NAME + " - RESTful API",
    version=settings.VERSION,
    description=app_description,
    openapi_tags=tags_metadata,
    docs_url="/docs" if settings.DOCS_ENABLED else None,
    redoc_url="/redoc" if settings.DOCS_ENABLED else None,
    openapi_url="/openapi.json" if settings.DOCS_ENABLED else None,
)

# CORS: ALLOWED_ORIGINS muhit o'zgaruvchisi bilan boshqariladi.
# "*" (lokal demo) bo'lsa credentials o'chiriladi — brauzerlar "*" + credentials kombinatsiyasini rad etadi.
_origins = settings.allowed_origins_list
app.add_middleware(
    CORSMiddleware,
    allow_origins=_origins,
    allow_credentials=(_origins != ["*"]),
    allow_methods=["GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"],
    allow_headers=["Authorization", "Content-Type", "X-API-Key", "X-Lesson-Stage", "X-AI-Locked"],
)

_MAX_BODY = settings.MAX_UPLOAD_MB * 1024 * 1024

_CSP = (
```

```
    "default-src 'self'; "
    "script-src 'self' 'unsafe-inline'; "
    "style-src 'self' 'unsafe-inline' https://fonts.googleapis.com; "
    "font-src 'self' data: https://fonts.gstatic.com; "
    "img-src 'self' data: blob: https:; "
    "media-src 'self' blob: data:; "
    "connect-src 'self' https:; "
    "frame-ancestors 'self'; "
    "base-uri 'self'; "
    "form-action 'self'; "
    "object-src 'none'"
)


@app.middleware("http")
async def security_and_cache_headers(request: Request, call_next):
    # Juda katta so'rov tanasi (DoS / xotira) — Content-Length bo'yicha oldindan rad etiladi
    cl = request.headers.get("content-length")
    if cl and cl.isdigit() and int(cl) > _MAX_BODY:
        return JSONResponse(status_code=413, content={
            "detail": f"So'rov hajmi {settings.MAX_UPLOAD_MB} MB dan oshmasligi kerak."
        })

    response = await call_next(request)
    path = request.url.path

    # Xavfsizlik sarlavhalari (nginx bo'lmasa ham ishlaydi)
    response.headers.setdefault("X-Content-Type-Options", "nosniff")
    response.headers.setdefault("X-Frame-Options", "SAMEORIGIN")
    response.headers.setdefault("Referrer-Policy", "strict-origin-when-cross-origin")
    response.headers.setdefault("Permissions-Policy", "camera=(), geolocation=(), payment=()")
    if not path.startswith(("/docs", "/redoc", "/openapi.json")):
        response.headers.setdefault("Content-Security-Policy", _CSP)

    # Frontend fayllari (HTML/JS/CSS) brauzerda keshlanib qolmasin — har yangilanish darhol ko'rinadi
    if not path.startswith(settings.API_V1_STR) and (
        path in ("/", "") or path.endswith((".html", ".js", ".css"))
    ):
        response.headers["Cache-Control"] = "no-cache, no-store, must-revalidate"
        response.headers["Pragma"] = "no-cache"
        response.headers["Expires"] = "0"
    elif path.startswith(settings.API_V1_STR):
        response.headers.setdefault("Cache-Control", "no-store")
    return response

# 1. API routerlarini ulash
app.include_router(auth_router, prefix=settings.API_V1_STR)
app.include_router(student_router, prefix=settings.API_V1_STR)
app.include_router(lesson_router, prefix=settings.API_V1_STR)
app.include_router(ai_router, prefix=settings.API_V1_STR)
app.include_router(teacher_router, prefix=settings.API_V1_STR)
app.include_router(teacher_diag_router, prefix=settings.API_V1_STR)
app.include_router(feedback_router, prefix=settings.API_V1_STR)
app.include_router(research_router, prefix=settings.API_V1_STR)
app.include_router(admin_router, prefix=settings.API_V1_STR)
app.include_router(proposals_router, prefix=settings.API_V1_STR)
app.include_router(api_keys_router, prefix=settings.API_V1_STR)
app.include_router(content_router, prefix=settings.API_V1_STR)
app.include_router(teacher_panel_router, prefix=settings.API_V1_STR)

# 2. Frontend statik fayllarni ulash
base_dir = os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
frontend_dir = os.path.realpath(os.path.join(base_dir, "frontend"))
css_dir = os.path.join(frontend_dir, "css")
js_dir = os.path.join(frontend_dir, "js")

if os.path.exists(css_dir):
    app.mount("/css", StaticFiles(directory=css_dir), name="css")
if os.path.exists(js_dir):
    app.mount("/js", StaticFiles(directory=js_dir), name="js")

# O'qituvchi yuklagan audio va boshqa fayllar (uploads/audio/...)
uploads_dir = os.path.join(base_dir, "uploads")
os.makedirs(os.path.join(uploads_dir, "audio"), exist_ok=True)
app.mount("/uploads", StaticFiles(directory=uploads_dir), name="uploads")

@app.get("/health")
def health_check():
    return {
        "status": "healthy",
        "app": settings.PROJECT_NAME,
        "version": settings.VERSION,
        "mode": "full_python_web_server",
        "rules_active": ["1_ai_lock", "2_rationale_transparent", "3_teacher_sovereignty", "4_student_reflection"]
    }


def _is_private_client(request: Request) -> bool:
    """Faqat lokal/ichki tarmoqdagi mijozlarga (Wi-Fi demo) ichki IP ma'lumotlari beriladi."""
    host = request.client.host if request.client else ""
    try:
```

```
        ip = ipaddress.ip_address(host)
        return ip.is_private or ip.is_loopback or ip.is_link_local
    except ValueError:
        return host in ("localhost", "testclient")


# 3. Tarmoq va Wi-Fi ma'lumotlari API (faqat lokal tarmoq mijozlari uchun)
@app.get("/api/network-info")
def get_network_info(request: Request):
    if not _is_private_client(request):
        return JSONResponse(status_code=403, content={"detail": "Tarmoq ma'lumotlari faqat lokal tarmoqda ko'rsatiladi."})
    import socket
    ips = []
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        s.settimeout(0.5)
        s.connect(('8.8.8.8', 80))
        primary = s.getsockname()[0]
        s.close()
        if primary and not primary.startswith('127.') and not primary.startswith('169.254.'):
            ips.append(primary)
    except Exception:
        pass
    try:
        for ip in socket.gethostbyname_ex(socket.gethostname())[2]:
            if not ip.startswith('127.') and not ip.startswith('169.254.') and ip not in ips:
                ips.append(ip)
    except Exception:
        pass
    primary_ip = ips[0] if ips else "127.0.0.1"
    return {
        "primary_ip": primary_ip,
        "all_ips": ips,
        "port": 8010,
        "lan_url": f"http://{primary_ip}:8010",
        "local_url": "http://127.0.0.1:8010",
        "qr_url": "/wifi_qr.png"
    }

# 4. Saytning asosiy yuzi (Python server orqali to'liq yetkaziladi)
@app.get("/")
def serve_home():
    index_file = os.path.join(frontend_dir, "index.html")
    if os.path.exists(index_file):
        return FileResponse(index_file)
    return {"message": "TILORA veb-platformasi ishlab turibdi", "docs": "/docs"}

# Noma'lum /api yo'llari: har qanday metod uchun aniq 404 (SPA fallback 405 bermasin)
@app.api_route("/api/{rest:path}", methods=["GET", "POST", "PUT", "PATCH", "DELETE"])
def api_not_found(rest: str):
    return JSONResponse(status_code=404, content={
        "detail": f"API endpoint topilmadi: /api/{rest}. Agar kod yangilangan bo'lsa, serverni qayta ishga tushiring (start.bat)."
    })

# SPA fallback va statik fayllarni yetkazish
@app.get("/{full_path:path}")
def serve_spa(full_path: str):
    # Agar API marshrut bo'lmasa, index.html ga yo'naltirish
    if full_path.startswith("api/") or full_path.startswith("docs") or full_path.startswith("openapi.json"):
        return JSONResponse(status_code=404, content={"error": "Not Found"})

    clean_path = full_path
    if clean_path.startswith("frontend/"):
        clean_path = clean_path[len("frontend/"):]

    # Path traversal himoyasi: faqat frontend papkasi ichidagi, nuqta bilan boshlanmaydigan fayllar
    target_file = os.path.realpath(os.path.join(frontend_dir, clean_path))
    inside = target_file == frontend_dir or target_file.startswith(frontend_dir + os.sep)
    hidden = any(part.startswith(".") for part in clean_path.replace("\\", "/").split("/") if part)
    if inside and not hidden and os.path.isfile(target_file):
        return FileResponse(target_file)
    index_file = os.path.join(frontend_dir, "index.html")
    if os.path.exists(index_file):
        return FileResponse(index_file)
    return {"message": "TILORA SPA"}

```

Fayl: backend/app/config.py

Sozlamalar: JWT kaliti, baza, xavfsizlik parametrlari

```
import os
import secrets
import warnings
from pydantic import BaseModel

_base_dir = os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
_default_db = f"sqlite:///{os.path.join(_base_dir, 'tilora.db').replace(chr(92), '/')}"


```

```python
def _load_dotenv() -> None:
    """Loyiha ildizidagi .env faylini (bo'lsa) muhitga yuklaydi; mavjud env qiymatlari ustuvor."""
    path = os.path.join(_base_dir, ".env")
    if not os.path.isfile(path):
        return
    try:
        with open(path, "r", encoding="utf-8") as f:
            for line in f:
                line = line.strip()
                if not line or line.startswith("#") or "=" not in line:
                    continue
                key, _, value = line.partition("=")
                key = key.strip()
                value = value.strip().strip('"').strip("'")
                if key and key not in os.environ:
                    os.environ[key] = value
    except OSError:
        pass


_load_dotenv()


def _resolve_secret_key() -> str:
    """
    JWT imzo kaliti:
      1. SECRET_KEY muhit o'zgaruvchisi (production uchun majburiy);
      2. bo'lmasa, loyiha ildizidagi .secret_key faylidan o'qiladi;
      3. u ham bo'lmasa, tasodifiy kalit yaratilib shu faylga yoziladi.
    Kodga yozilgan doimiy kalit ishlatilmaydi (token soxtalashtirish xavfi).
    """
    env_key = os.getenv("SECRET_KEY", "").strip()
    placeholders = {"", "change_this_to_a_random_32_character_secret_string", "generate_random_secret_key_here",
                    "tilora-secret-key-default"}
    if env_key and env_key not in placeholders and len(env_key) >= 32:
        return env_key
    if env_key and env_key in placeholders:
        warnings.warn("SECRET_KEY namunaviy qiymatda qoldirilgan — avtomatik yaratilgan kalit ishlatiladi.")
    key_file = os.path.join(_base_dir, ".secret_key")
    try:
        if os.path.isfile(key_file):
            with open(key_file, "r", encoding="utf-8") as f:
                saved = f.read().strip()
            if len(saved) >= 32:
                return saved
        generated = secrets.token_urlsafe(48)
        with open(key_file, "w", encoding="utf-8") as f:
            f.write(generated)
        return generated
    except OSError:
        # Fayl tizimi yozib bo'lmaydigan joy (masalan, read-only konteyner): vaqtinchalik kalit
        warnings.warn("SECRET_KEY saqlab bo'lmadi — server qayta ishga tushganda sessiyalar tugaydi.")
        return secrets.token_urlsafe(48)


def _resolve_database_url() -> str:
    """DATABASE_URL: SQLite yo'li mavjud bo'lmagan papkaga ko'rsatsa (boshqa kompyuterdan ko'chirilgan .env), standart yo'l."""
    url = os.getenv("DATABASE_URL", "").strip()
    if not url:
        return _default_db
    if url.startswith("sqlite:///"):
        raw_path = url[len("sqlite:///"):]
        if raw_path and raw_path != ":memory:":
            parent = os.path.dirname(os.path.abspath(raw_path))
            if not os.path.isdir(parent):
                warnings.warn(f"DATABASE_URL papkasi topilmadi ({parent}); standart SQLite bazasi ishlatiladi.")
                return _default_db
    return url


def _env_bool(name: str, default: bool) -> bool:
    raw = os.getenv(name)
    if raw is None:
        return default
    return raw.strip().lower() not in ("0", "false", "no", "off")


class Settings(BaseModel):
    PROJECT_NAME: str = "TILORA Adaptiv Ta'lim Platformasi"
    VERSION: str = "2.2.0"
    API_V1_STR: str = "/api"
    SECRET_KEY: str = _resolve_secret_key()
    ALGORITHM: str = "HS256"
    ACCESS_TOKEN_EXPIRE_MINUTES: int = int(os.getenv("ACCESS_TOKEN_EXPIRE_MINUTES", str(60 * 12)))  # 12 soat
    REFRESH_TOKEN_EXPIRE_DAYS: int = 7
    DATABASE_URL: str = _resolve_database_url()

    # AI kalitlari faqat muhit o'zgaruvchisidan olinadi; kodda standart kalit yo'q.
    AI_API_KEY: str = os.getenv("AI_API_KEY", os.getenv("OPENAI_API_KEY", "")).strip()
    AI_BASE_URL: str = os.getenv("AI_BASE_URL", os.getenv("OPENAI_BASE_URL", "https://api.b.ai/v1")).rstrip("/")
    AI_MODEL: str = os.getenv("AI_MODEL", "deepseek-v4-flash-vision-exp")
```

```
    ANTHROPIC_API_KEY: str = os.getenv("ANTHROPIC_API_KEY", "")
    OPENAI_API_KEY: str = os.getenv("OPENAI_API_KEY", "").strip()
    DEFAULT_LOCALE: str = "uz"
    # Faqat oxirgi chora: AI_SSL_VERIFY=false bo'lsa AI API uchun sertifikat tekshiruvi o'chiriladi
    AI_SSL_VERIFY: bool = _env_bool("AI_SSL_VERIFY", True)

    # Xavfsizlik sozlamalari
    PASSWORD_MIN_LENGTH: int = int(os.getenv("PASSWORD_MIN_LENGTH", "8"))
    # CORS: vergul bilan ajratilgan ro'yxat; "*" faqat lokal demo uchun (credentials bilan birga ishlatilmaydi)
    ALLOWED_ORIGINS: str = os.getenv("ALLOWED_ORIGINS", "*")
    DOCS_ENABLED: bool = _env_bool("DOCS_ENABLED", True)
    MAX_UPLOAD_MB: int = int(os.getenv("MAX_UPLOAD_MB", "25"))
    # Login urinishlari: bitta IP/login uchun 15 daqiqada ruxsat etilgan xato urinishlar soni
    LOGIN_MAX_ATTEMPTS: int = int(os.getenv("LOGIN_MAX_ATTEMPTS", "10"))
    LOGIN_WINDOW_SECONDS: int = int(os.getenv("LOGIN_WINDOW_SECONDS", "900"))
    # AI so'rovlari: bitta foydalanuvchi uchun 10 daqiqada
    AI_MAX_REQUESTS: int = int(os.getenv("AI_MAX_REQUESTS", "60"))
    AI_WINDOW_SECONDS: int = int(os.getenv("AI_WINDOW_SECONDS", "600"))
    # Demo (seed) hisoblar paroli — faqat seed.py uchun
    SEED_PASSWORD: str = os.getenv("SEED_PASSWORD", "<REDACTED>")

    @property
    def allowed_origins_list(self):
        raw = (self.ALLOWED_ORIGINS or "").strip()
        if not raw or raw == "*":
            return ["*"]
        return [o.strip() for o in raw.split(",") if o.strip()]


settings = Settings()

```

Fayl: backend/app/database.py

SQLAlchemy ulanishi va sessiya

```
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker
from app.config import settings

connect_args = {"check_same_thread": False} if settings.DATABASE_URL.startswith("sqlite") else {}

engine = create_engine(
    settings.DATABASE_URL,
    connect_args=connect_args
)

SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

```

Fayl: backend/app/core/security.py

Parol xeshlash (PBKDF2-SHA256) va JWT (HMAC-SHA256)

```
"""
Parol xeshlash va JWT token.

Parol formati (yangi): ``pbkdf2_sha256$<iteratsiya>$<tuz_hex>$<xesh_hex>``
  - har bir foydalanuvchi uchun alohida tasodifiy tuz (16 bayt);
  - 600 000 iteratsiya (OWASP 2023 tavsiyasi).
Eski format (64 ta hex belgi, umumiy statik tuz) faqat tekshirish uchun qo'llab-quvvatlanadi
va foydalanuvchi muvaffaqiyatli kirganda avtomatik yangi formatga o'tkaziladi (``needs_rehash``).
"""
import hashlib
import hmac
import base64
import json
import secrets
import time
from datetime import datetime, timedelta, timezone
from typing import Optional, Dict, Any
from app.config import settings

PBKDF2_ITERATIONS = 600_000
_LEGACY_SALT = "<REDACTED>"
_LEGACY_ITERATIONS = 100_000


def _pbkdf2(password: str, salt: bytes, iterations: int) -> bytes:
    return hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt, iterations)

```

```python

def hash_password(password: str) -> str:
    """Xavfsiz parol xeshlash (PBKDF2-HMAC-SHA256, foydalanuvchiga xos tuz)."""
    salt = secrets.token_bytes(16)
    key = _pbkdf2(password, salt, PBKDF2_ITERATIONS)
    return f"pbkdf2_sha256${PBKDF2_ITERATIONS}${salt.hex()}${key.hex()}"


def _is_legacy_hash(hashed: str) -> bool:
    return len(hashed) == 64 and all(c in "0123456789abcdef" for c in hashed.lower())


def verify_password(plain_password: str, hashed_password: str) -> bool:
    """Parolni tekshirish (doimiy vaqtli taqqoslash)."""
    if not plain_password or not hashed_password:
        return False
    try:
        if hashed_password.startswith("pbkdf2_sha256$"):
            _, iters, salt_hex, key_hex = hashed_password.split("$", 3)
            computed = _pbkdf2(plain_password, bytes.fromhex(salt_hex), int(iters))
            return hmac.compare_digest(computed, bytes.fromhex(key_hex))
        if _is_legacy_hash(hashed_password):
            computed = _pbkdf2(plain_password, _LEGACY_SALT.encode("utf-8"), _LEGACY_ITERATIONS)
            return hmac.compare_digest(computed.hex(), hashed_password.lower())
    except (ValueError, TypeError):
        return False
    return False


def needs_rehash(hashed_password: str) -> bool:
    """Eski formatdagi yoki kam iteratsiyali xeshni yangilash kerakmi."""
    if not hashed_password or not hashed_password.startswith("pbkdf2_sha256$"):
        return True
    try:
        iters = int(hashed_password.split("$", 2)[1])
        return iters < PBKDF2_ITERATIONS
    except (ValueError, IndexError):
        return True


def _b64url(data: bytes) -> str:
    return base64.urlsafe_b64encode(data).decode().rstrip("=")


def _b64url_decode(s: str) -> bytes:
    return base64.urlsafe_b64decode(s + "=" * (-len(s) % 4))


def create_access_token(data: dict, expires_delta: Optional[timedelta] = None) -> str:
    """HS256 bilan imzolangan JWT (iat, exp, jti bilan)."""
    to_encode = data.copy()
    now = datetime.now(timezone.utc)
    expire = now + (expires_delta or timedelta(minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES))
    to_encode.update({"iat": int(now.timestamp()), "exp": int(expire.timestamp()), "jti": secrets.token_hex(8)})
    header = {"alg": "HS256", "typ": "JWT"}

    h_b64 = _b64url(json.dumps(header, separators=(",", ":")).encode())
    p_b64 = _b64url(json.dumps(to_encode, separators=(",", ":")).encode())
    message = f"{h_b64}.{p_b64}"
    signature = hmac.new(settings.SECRET_KEY.encode(), message.encode(), hashlib.sha256).digest()
    return f"{message}.{_b64url(signature)}"


def decode_access_token(token: str) -> Optional[Dict[str, Any]]:
    """Imzoni, algoritmni va amal qilish muddatini tekshirib tokenni ochish."""
    try:
        parts = token.split(".")
        if len(parts) != 3:
            return None
        h_b64, p_b64, s_b64 = parts
        header = json.loads(_b64url_decode(h_b64).decode())
        if header.get("alg") != "HS256":
            return None  # "none" yoki boshqa algoritm qabul qilinmaydi

        message = f"{h_b64}.{p_b64}"
        expected_sig = hmac.new(settings.SECRET_KEY.encode(), message.encode(), hashlib.sha256).digest()
        actual_sig = _b64url_decode(s_b64)
        if not hmac.compare_digest(expected_sig, actual_sig):
            return None

        payload = json.loads(_b64url_decode(p_b64).decode())
        exp = payload.get("exp")
        if not isinstance(exp, (int, float)) or exp < int(time.time()):
            return None
        return payload
    except Exception:
        return None

```

Fayl: backend/app/core/deps.py

Avtorizatsiya bog'liqliklari: joriy foydalanuvchi, rollar

```python
import hashlib
from datetime import datetime, timezone
from typing import Optional, List
from fastapi import Depends, Header, HTTPException, Request, status
from app.core.security import decode_access_token
from app.core.ratelimit import SlidingWindowLimiter

# API kalitlar uchun daqiqalik chegara (kalitdagi rate_limit maydoni asosida)
_api_key_limiter = SlidingWindowLimiter(max_events=120, window_seconds=60)


def _utcnow_naive() -> datetime:
    return datetime.now(timezone.utc).replace(tzinfo=None)


def _refresh_from_db(payload: dict) -> dict:
    """
    JWT ichidagi rol/obuna ma'lumotlari eskirib qolishi mumkin (admin o'zgartirgan bo'lsa).
    Foydalanuvchi bazada topilsa, dolzarb qiymatlar olinadi; nofaol yoki tasdiqlanmagan
    hisob uchun token rad etiladi. Bazada topilmasa (masalan, test tokenlari) payload o'zgarmaydi.
    """
    uid = payload.get("id") or payload.get("sub")
    if not uid:
        return payload
    from app.database import SessionLocal
    from app.models.entities import User

    db = SessionLocal()
    try:
        user = db.query(User).filter(User.id == uid).first()
    finally:
        db.close()
    if not user:
        return payload
    if not bool(getattr(user, "is_active", True)):
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Hisob nofaol holatda. Administratorga murojaat qi
ling.")
    if not bool(getattr(user, "is_approved", True)) and user.role != "admin":
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Hisob hali administrator tomonidan tasdiqlanmagan
.")
    payload = dict(payload)
    payload.update({
        "id": user.id,
        "email": user.email,
        "role": user.role,
        "full_name": user.full_name,
        "institution_id": user.institution_id,
        "group_id": user.group_id,
        "subscription_tier": getattr(user, "subscription_tier", "free") or "free",
        "is_approved": bool(getattr(user, "is_approved", True)),
    })
    return payload


async def get_current_user(
    authorization: Optional[str] = Header(None),
    x_api_key: Optional[str] = Header(None, alias="X-API-Key")
) -> dict:
    """
    Joriy foydalanuvchini olish (2 ta xavfsiz usul):
    1. HTTP Bearer JWT token: '<REDACTED>'
    2. Dasturchi/Admin API Kaliti: 'X-API-Key: tilora_sk_live_...' yoki 'Authorization: ApiKey <key>'
    """
    raw_api_key = None
    if x_api_key and x_api_key.strip():
        raw_api_key = x_api_key.strip()
    elif authorization and authorization.strip().startswith("ApiKey "):
        raw_api_key = authorization.replace("ApiKey ", "").strip()

    # 1-usul: API Kalit orqali autentifikatsiya
    if raw_api_key:
        from app.database import SessionLocal
        from app.models.entities import ApiKey, User

        key_hash = hashlib.sha256(raw_api_key.encode("utf-8")).hexdigest()
        db = SessionLocal()
        try:
            api_key_obj = db.query(ApiKey).filter(ApiKey.key_hash == key_hash).first()
            if not api_key_obj or not api_key_obj.is_active:
                raise HTTPException(
                    status_code=status.HTTP_401_UNAUTHORIZED,
                    detail="Yaroqsiz yoki bekor qilingan API kalit (Invalid or revoked API key)."
                )

            if api_key_obj.expires_at and api_key_obj.expires_at < _utcnow_naive():
                raise HTTPException(
                    status_code=status.HTTP_401_UNAUTHORIZED,
                    detail="API kalitning amal qilish muddati tugagan (API key has expired)."
```

```
                )

            user = db.query(User).filter(User.id == api_key_obj.user_id).first()
            if not user or not user.is_active:
                raise HTTPException(
                    status_code=status.HTTP_401_UNAUTHORIZED,
                    detail="API kalit egasi bo'lgan foydalanuvchi hisobi faol emas."
                )

            # Kalitga belgilangan daqiqalik chegara
            per_minute = int(api_key_obj.rate_limit or 120)
            limiter = _api_key_limiter if per_minute == 120 else SlidingWindowLimiter(per_minute, 60)
            limiter.check_and_record(
                f"apikey:{api_key_obj.id}",
                f"API kalit uchun so'rovlar chegarasi oshdi (daqiqasiga {per_minute} ta). Biroz kuting."
            )

            # Oxirgi ishlatilgan vaqtni yangilash
            api_key_obj.last_used_at = _utcnow_naive()
            db.commit()

            return {
                "id": user.id,
                "email": user.email,
                "role": api_key_obj.role or user.role,
                "user_role": user.role,
                "full_name": user.full_name,
                "institution_id": user.institution_id,
                "group_id": user.group_id,
                "subscription_tier": getattr(user, "subscription_tier", "pro") or "pro",
                "api_key_id": api_key_obj.id,
                "api_key_name": api_key_obj.name,
                "auth_type": "api_key"
            }
        finally:
            db.close()

    # 2-usul: JWT Bearer token orqali autentifikatsiya
    if not authorization:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Tizimga kirilmagan. Iltimos, JWT token (Bearer) yoki API kalit (X-API-Key) yuboring."
        )

    token = authorization.replace("Bearer ", "").strip()
    payload = decode_access_token(token)
    if not payload:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Autentifikatsiya muddati tugagan yoki token yaroqsiz."
        )
    payload = _refresh_from_db(payload)
    payload["auth_type"] = "jwt"
    return payload


def require_role(allowed_roles: List[str]):
    """Rolni qat'iy tekshirish dependency'si (RBAC)."""
    async def role_checker(current_user: dict = Depends(get_current_user)):
        user_role = current_user.get("role")
        if user_role not in allowed_roles:
            raise HTTPException(
                status_code=status.HTTP_403_FORBIDDEN,
                detail=f"Kirish taqiqlangan: Ushbu amal faqat {', '.join(allowed_roles)} rollari uchun ruxsat etilgan. Sizning
    rolingiz: {user_role}"
            )
        return current_user
    return role_checker


async def get_optional_user(
    authorization: Optional[str] = Header(None),
    x_api_key: Optional[str] = Header(None, alias="X-API-Key")
) -> Optional[dict]:
    """Token yoki API kalit bo'lsa foydalanuvchini olish, bo'lmasa None qaytarish (mehmon rejimi)."""
    try:
        return await get_current_user(authorization=authorization, x_api_key=x_api_key)
    except HTTPException:
        return None

```

Fayl: backend/app/core/access.py

Tarif (obuna) bo'yicha kirish tekshiruvi

```
from fastapi import Request, status
from app.core.errors import TiloraException

def check_experiment_access(user_arm: str, feature: str):
    """
    Tadqiqot butunligi: Nazorat guruhidagi talabaga adaptiv yo'l va
    metakognitiv blok ko'rsatilmaydi.
    """
    if user_arm == "control" and feature in ["adaptive_path", "metacognitive_block", "ai_scaffold"]:
        raise TiloraException(
            code="CONTROL_GROUP_RESTRICTED",
            status_code=status.HTTP_403_FORBIDDEN,
            details={"experiment_arm": "control", "feature": feature}
        )
    return True

```

Fayl: backend/app/core/ratelimit.py

Kirish urinishlarini cheklash (rate limiting)

```
"""
Oddiy xotiradagi (in-memory) so'rovlar chegaralovchisi.

Bitta jarayon ichida ishlaydi (gunicorn bir nechta worker bilan ishlasa, har worker alohida
hisoblaydi — bu holda nginx `limit_req` qo'shimcha himoya beradi, qarang nginx/conf.d/default.conf).
"""
import threading
import time
from collections import deque
from typing import Deque, Dict, Tuple

from fastapi import HTTPException, Request, status


class SlidingWindowLimiter:
    def __init__(self, max_events: int, window_seconds: int):
        self.max_events = max(1, int(max_events))
        self.window = max(1, int(window_seconds))
        self._events: Dict[str, Deque[float]] = {}
        self._lock = threading.Lock()

    def _prune(self, key: str, now: float) -> Deque[float]:
        q = self._events.setdefault(key, deque())
        cutoff = now - self.window
        while q and q[0] < cutoff:
            q.popleft()
        return q

    def is_limited(self, key: str) -> Tuple[bool, int]:
        """(cheklanganmi, qancha soniyadan keyin ochiladi)."""
        now = time.time()
        with self._lock:
            q = self._prune(key, now)
            if len(q) >= self.max_events:
                retry = int(q[0] + self.window - now) + 1
                return True, max(1, retry)
            return False, 0

    def record(self, key: str) -> None:
        now = time.time()
        with self._lock:
            q = self._prune(key, now)
            q.append(now)
            # Xotira o'sib ketmasligi uchun eski kalitlarni vaqti-vaqti bilan tozalash
            if len(self._events) > 5000:
                for k in [k for k, v in self._events.items() if not v or v[-1] < now - self.window]:
                    self._events.pop(k, None)

    def reset(self, key: str) -> None:
        with self._lock:
            self._events.pop(key, None)

    def check_and_record(self, key: str, detail: str) -> None:
        """Cheklov oshgan bo'lsa 429 beradi, aks holda hodisani yozadi."""
        limited, retry = self.is_limited(key)
        if limited:
            raise HTTPException(
                status_code=status.HTTP_429_TOO_MANY_REQUESTS,
                detail=detail,
                headers={"Retry-After": str(retry)},
            )
        self.record(key)


def client_ip(request: Request) -> str:
    """Proksi orqasida bo'lsa X-Forwarded-For ning birinchi manzili, aks holda to'g'ridan-to'g'ri IP."""
    fwd = request.headers.get("x-forwarded-for")
```

```
    if fwd:
        return fwd.split(",")[0].strip()[:64]
    real = request.headers.get("x-real-ip")
    if real:
        return real.strip()[:64]
    return (request.client.host if request.client else "unknown")[:64]

```

Fayl: backend/app/core/errors.py

Xatolarni yagona formatda qaytarish

```
from fastapi import HTTPException, status

ERROR_MESSAGES = {
    "AI_LOCKED": {
        "uz": "Mustaqil bajarish bosqichida sun'iy intellekt vositalaridan foydalanish taqiqlangan. Bu ko'nikmani shakllantirishning asosiy pedagogik qoidasidir.",
        "en": "AI assistance is locked during the independent stage to foster authentic skill acquisition.",
        "ru": "Использование ИИ заблокировано на этапе самостоятельной работы."
    },
    "SELF_RATING_REQUIRED": {
        "uz": "Haqiqiy natijani ko'rish uchun avval o'z baholashingizni (metakognitiv mezonlar bo'yicha) saqlashingiz shart.",
        "en": "Self-rating must be submitted before viewing the actual objective rating.",
        "ru": "Сначала сохраните самооценку перед просмотром объективного результата."
    },
    "NOT_FOUND": {
        "uz": "So'ralgan resurs topilmadi.",
        "en": "Requested resource not found.",
        "ru": "Запрашиваемый ресурс не найден."
    },
    "INVALID_CREDENTIALS": {
        "uz": "Email yoki parol noto'g'ri.",
        "en": "Invalid email or password.",
        "ru": "Неверный адрес электронной почты или пароль."
    },
    "FORBIDDEN_ROLE": {
        "uz": "Ushbu amalni bajarish uchun rolingiz yetarli emas.",
        "en": "Insufficient role permissions for this operation.",
        "ru": "Недостаточно прав для выполнения действия."
    },
    "CONTROL_GROUP_RESTRICTED": {
        "uz": "Nazorat guruhi a'zolariga adaptiv va metakognitiv modullarga kirish taqiqlangan (ilmiy eksperiment dizayni).",
        "en": "Control group members have restricted access to adaptive modules (research integrity).",
        "ru": "Членам контрольной группы закрыт доступ к адаптивным модулям."
    },
    "PROPOSAL_ALREADY_DECIDED": {
        "uz": "Ushbu taklif allaqachon ko'rib chiqilgan.",
        "en": "This proposal has already been decided.",
        "ru": "Это предложение уже рассмотрено."
    }
}

class TiloraException(HTTPException):
    def __init__(self, code: str, status_code: int = status.HTTP_400_BAD_REQUEST, locale: str = "uz", details: dict = None):
        msg_dict = ERROR_MESSAGES.get(code, {"uz": code, "en": code, "ru": code})
        detail_msg = msg_dict.get(locale, msg_dict["uz"])
        payload = {
            "error_code": code,
            "message": detail_msg,
            "details": details or {}
        }
        super().__init__(status_code=status_code, detail=payload)

```

Fayl: backend/app/models/entities.py

Ma'lumotlar bazasi modellari (User, Group, AuditLog va b.)

```
import uuid
from datetime import datetime
from sqlalchemy import (
    Column, String, Integer, SmallInteger, Boolean, Text, DateTime, ForeignKey, JSON, Float
)
from sqlalchemy.orm import relationship
from app.database import Base

def gen_uuid():
    return str(uuid.uuid4())

class Institution(Base):
    __tablename__ = "institutions"
    id = Column(String(36), primary_key=True, default=gen_uuid)
    name = Column(String(255), nullable=False)
    short_name = Column(String(50), nullable=False)
    direction_code = Column(String(20), default="60110900")
    created_at = Column(DateTime, default=datetime.utcnow)

class Group(Base):
    __tablename__ = "groups"
```

```
    id = Column(String(36), primary_key=True, default=gen_uuid)
    institution_id = Column(String(36), ForeignKey("institutions.id"))
    name = Column(String(50), nullable=False)
    course_year = Column(SmallInteger, default=3)
    teacher_id = Column(String(36), nullable=True)
    experiment_arm = Column(String(20), default="experimental") # experimental, control, none
    created_at = Column(DateTime, default=datetime.utcnow)

class User(Base):
    __tablename__ = "users"
    id = Column(String(36), primary_key=True, default=gen_uuid)
    email = Column(String(255), unique=True, index=True, nullable=False)
    password_hash = Column(String(255), nullable=False)
    full_name = Column(String(255), nullable=False)
    role = Column(String(20), default="student") # student, teacher, researcher, admin
    institution_id = Column(String(36), ForeignKey("institutions.id"), nullable=True)
    group_id = Column(String(36), ForeignKey("groups.id"), nullable=True)
    locale = Column(String(5), default="uz")
    is_active = Column(Boolean, default=True)
    is_approved = Column(Boolean, default=False) # Super admin tasdiqlashi lozim
    subscription_tier = Column(String(20), default="free") # "free", "pro"
    subscription_status = Column(String(30), default="active") # "active", "pending", "expired"
    phone_number = Column(String(30), nullable=True)
    created_at = Column(DateTime, default=datetime.utcnow)

class StudentProfile(Base):
    """3 o'lchovli profil (tarix saqlanadi, qator o'chirilmaydi)."""
    __tablename__ = "student_profiles"
    id = Column(String(36), primary_key=True, default=gen_uuid)
    student_id = Column(String(36), ForeignKey("users.id"), index=True, nullable=False)
    cefr_listening = Column(String(2), default="B1")
    cefr_reading = Column(String(2), default="B2")
    cefr_speaking = Column(String(2), default="A2")
    cefr_writing = Column(String(2), default="B1")
    professional_track = Column(String(100), default="Filologiya va tillarni o'qitish (ingliz tili)")
    load_capacity = Column(SmallInteger, default=50) # 1-100
    pace_index = Column(SmallInteger, default=50)    # 1-100
    error_persistence = Column(SmallInteger, default=50) # 1-100
    anxiety_level = Column(SmallInteger, default=70) # 1-100 (FLCAS)
    motivation_type = Column(String(20), default="intrinsic")
    autonomy_readiness = Column(SmallInteger, default=40)
    source = Column(String(30), default="initial_test") # initial_test, auto_update, teacher_override
    valid_from = Column(DateTime, default=datetime.utcnow)

class LearningPath(Base):
    __tablename__ = "learning_paths"
    id = Column(String(36), primary_key=True, default=gen_uuid)
    student_id = Column(String(36), ForeignKey("users.id"), index=True, nullable=False)
    profile_id = Column(String(36), ForeignKey("student_profiles.id"), nullable=True)
    status = Column(String(20), default="active") # active, completed, revised
    revision_reason = Column(Text, nullable=True)
    created_at = Column(DateTime, default=datetime.utcnow)

class PathItem(Base):
    __tablename__ = "path_items"
    id = Column(String(36), primary_key=True, default=gen_uuid)
    path_id = Column(String(36), ForeignKey("learning_paths.id"), nullable=False)
    week_number = Column(SmallInteger, nullable=False)
    skill = Column(String(20), nullable=False)
    topic = Column(String(255), nullable=False)
    difficulty_level = Column(SmallInteger, default=5) # 1-10
    scaffold_level = Column(String(20), default="full") # full, partial, none
    rationale = Column(Text, nullable=False) # 2-QOIDA: bo'sh bo'lishi mumkin emas
    status = Column(String(20), default="planned") # planned, current, done
    teacher_approved = Column(Boolean, default=True)
    approved_by = Column(String(36), nullable=True)
    approved_at = Column(DateTime, nullable=True)

class Lesson(Base):
    __tablename__ = "lessons"
    id = Column(String(36), primary_key=True, default=gen_uuid)
    path_item_id = Column(String(36), ForeignKey("path_items.id"), nullable=False)
    student_id = Column(String(36), ForeignKey("users.id"), nullable=False)
    started_at = Column(DateTime, default=datetime.utcnow)
    finished_at = Column(DateTime, nullable=True)
    status = Column(String(20), default="in_progress") # in_progress, completed
    final_score = Column(SmallInteger, nullable=True)

class LessonStage(Base):
    __tablename__ = "lesson_stages"
    id = Column(String(36), primary_key=True, default=gen_uuid)
    lesson_id = Column(String(36), ForeignKey("lessons.id"), nullable=False)
    stage = Column(String(30), nullable=False) # intro, preparation, independent, analysis, reflection
    started_at = Column(DateTime, default=datetime.utcnow)
    finished_at = Column(DateTime, nullable=True)
    ai_locked = Column(Boolean, default=False)
    duration_seconds = Column(Integer, default=0)

class StudentProduction(Base):
    __tablename__ = "student_productions"
    id = Column(String(36), primary_key=True, default=gen_uuid)
```

```
    lesson_id = Column(String(36), ForeignKey("lessons.id"), nullable=False)
    skill = Column(String(20), default="writing")
    content_text = Column(Text, nullable=True)
    audio_url = Column(String(255), nullable=True)
    word_count = Column(Integer, default=0)
    ai_assisted = Column(Boolean, default=False) # HAR DOIM False
    created_at = Column(DateTime, default=datetime.utcnow)

class FeedbackItem(Base):
    __tablename__ = "feedback_items"
    id = Column(String(36), primary_key=True, default=gen_uuid)
    production_id = Column(String(36), ForeignKey("student_productions.id"), nullable=False)
    category = Column(String(30), nullable=False) # grammar, lexis, coherence, pronunciation, content
    issue_short = Column(String(255), nullable=False)
    issue_reason = Column(Text, nullable=False) # Sabab — majburiy
    suggestion = Column(Text, nullable=True)
    l1_interference = Column(Boolean, default=False)
    student_accepts = Column(Boolean, nullable=True)
    student_note = Column(Text, nullable=True)
    teacher_extension = Column(Text, nullable=True)
    feedback_level = Column(String(30), default="task") # task, process, self_regulation

class MetacognitiveEntry(Base):
    __tablename__ = "metacognitive_entries"
    id = Column(String(36), primary_key=True, default=gen_uuid)
    lesson_id = Column(String(36), ForeignKey("lessons.id"), nullable=False)
    self_rating = Column(JSON, nullable=True)
    actual_rating = Column(JSON, nullable=True)
    gap_explanation = Column(Text, nullable=True)
    next_goal = Column(Text, nullable=True)
    created_at = Column(DateTime, default=datetime.utcnow)

class TeacherProposal(Base):
    __tablename__ = "teacher_proposals"
    id = Column(String(36), primary_key=True, default=gen_uuid)
    student_id = Column(String(36), ForeignKey("users.id"), nullable=True)
    teacher_id = Column(String(36), ForeignKey("users.id"), nullable=True)
    sender_id = Column(String(36), ForeignKey("users.id"), nullable=True)
    receiver_id = Column(String(36), ForeignKey("users.id"), nullable=True)
    proposal_type = Column(String(50), nullable=False) # difficulty_change, scaffold_change, deadline_change, path_revision, me
    thodology, assignment, peer_learning, general
    proposed_change = Column(JSON, nullable=False)
    basis = Column(Text, nullable=False) # Qaysi ko'rsatkichga tayanilgani yoki taklif maqsadi
    status = Column(String(20), default="pending") # pending, approved, modified, rejected
    teacher_comment = Column(Text, nullable=True)
    decided_at = Column(DateTime, nullable=True)
    created_at = Column(DateTime, default=datetime.utcnow)

class ResearchConsent(Base):
    __tablename__ = "research_consents"
    id = Column(String(36), primary_key=True, default=gen_uuid)
    user_id = Column(String(36), ForeignKey("users.id"), nullable=False)
    consent_given = Column(Boolean, default=True)
    consent_text_version = Column(String(20), default="1.0")
    given_at = Column(DateTime, default=datetime.utcnow)
    withdrawn_at = Column(DateTime, nullable=True)

class Assessment(Base):
    __tablename__ = "assessments"
    id = Column(String(36), primary_key=True, default=gen_uuid)
    student_id = Column(String(36), ForeignKey("users.id"), nullable=False)
    assessment_type = Column(String(20), default="pre") # pre, mid, post
    skill = Column(String(20), nullable=False)
    score = Column(SmallInteger, nullable=False)
    max_score = Column(SmallInteger, default=100)
    administered_at = Column(DateTime, default=datetime.utcnow)
    is_frozen = Column(Boolean, default=False)
    ai_tools_allowed = Column(Boolean, default=False)

class AuditLog(Base):
    __tablename__ = "audit_log"
    id = Column(String(36), primary_key=True, default=gen_uuid)
    actor_id = Column(String(36), nullable=False)
    action = Column(String(100), nullable=False)
    entity_type = Column(String(50), nullable=False)
    entity_id = Column(String(36), nullable=False)
    payload = Column(JSON, nullable=True)
    ip = Column(String(45), nullable=True)
    created_at = Column(DateTime, default=datetime.utcnow)

class TeacherSubject(Base):
    """O'qituvchiga biriktirilgan fanlar va sillabuslar."""
    __tablename__ = "teacher_subjects"
    id = Column(String(36), primary_key=True, default=gen_uuid)
    teacher_id = Column(String(36), ForeignKey("users.id"), index=True, nullable=False)
    name = Column(String(255), nullable=False)
    code = Column(String(50), nullable=True)
    semester = Column(String(50), default="5-semestr")
    course_year = Column(SmallInteger, default=3)
    group_name = Column(String(100), default="302-guruh")
    description = Column(Text, nullable=True)
```

```
    syllabus_filename = Column(String(255), nullable=True)
    syllabus_text = Column(Text, nullable=True)
    created_at = Column(DateTime, default=datetime.utcnow)

class SyllabusTopic(Base):
    """Sillabus ichidagi haftalik mavzular."""
    __tablename__ = "syllabus_topics"
    id = Column(String(36), primary_key=True, default=gen_uuid)
    subject_id = Column(String(36), ForeignKey("teacher_subjects.id"), index=True, nullable=False)
    week_number = Column(SmallInteger, default=1)
    title = Column(String(255), nullable=False)
    hours = Column(SmallInteger, default=2)
    objectives = Column(Text, nullable=True)
    created_at = Column(DateTime, default=datetime.utcnow)

class TeacherMaterial(Base):
    """O'qituvchi tomonidan AI yordamida yaratilgan test, prezentatsiya va dars rejalari."""
    __tablename__ = "teacher_materials"
    id = Column(String(36), primary_key=True, default=gen_uuid)
    teacher_id = Column(String(36), ForeignKey("users.id"), index=True, nullable=False)
    subject_id = Column(String(36), ForeignKey("teacher_subjects.id"), index=True, nullable=False)
    topic_id = Column(String(36), nullable=True)
    topic_title = Column(String(255), nullable=False)
    material_type = Column(String(50), nullable=False)  # "test", "presentation", "lesson_plan", "writing"
    title = Column(String(255), nullable=False)
    content_json = Column(JSON, nullable=True)
    content_text = Column(Text, nullable=True)
    created_at = Column(DateTime, default=datetime.utcnow)


class MaterialAttempt(Base):
    """Talabaning mustaqil topshiriq (test, yozma ish, slayd/reja ko'rish) natijasi.

    Har bir urinish alohida qator: test uchun ball va javoblar, yozma ish uchun matn va
    AI rubrika bahosi, slayd/dars rejasi uchun "ko'rildi" belgisi. O'qituvchi shu
    jadvaldan guruh natijalarini ko'radi.
    """
    __tablename__ = "material_attempts"
    id = Column(String(36), primary_key=True, default=gen_uuid)
    student_id = Column(String(36), ForeignKey("users.id"), index=True, nullable=False)
    material_id = Column(String(36), ForeignKey("teacher_materials.id"), index=True, nullable=False)
    subject_id = Column(String(36), ForeignKey("teacher_subjects.id"), index=True, nullable=True)
    teacher_id = Column(String(36), ForeignKey("users.id"), index=True, nullable=True)
    material_type = Column(String(50), nullable=False)
    status = Column(String(20), default="completed")  # completed | viewed
    score = Column(Integer, nullable=True)            # foiz (0-100) yoki rubrika bali foizda
    correct_count = Column(Integer, nullable=True)
    total_count = Column(Integer, nullable=True)
    passed = Column(Boolean, nullable=True)
    answers_json = Column(JSON, nullable=True)        # test javoblari yoki yozma matn
    result_json = Column(JSON, nullable=True)         # tekshiruv/rubrika natijasi
    created_at = Column(DateTime, default=datetime.utcnow, index=True)

class SubscriptionPayment(Base):
    """Foydalanuvchilarning PRO tarifiga to'lov arizalari (Click, Payme, Uzum)."""
    __tablename__ = "subscription_payments"
    id = Column(String(36), primary_key=True, default=gen_uuid)
    user_id = Column(String(36), ForeignKey("users.id"), index=True, nullable=False)
    user_name = Column(String(255), nullable=False)
    user_email = Column(String(255), nullable=False)
    role = Column(String(20), default="student")
    plan = Column(String(50), default="pro_monthly")  # "pro_monthly", "pro_annual"
    amount = Column(Integer, default=49000)  # so'm
    payment_method = Column(String(50), default="click")  # click, payme, card
    transaction_id = Column(String(100), nullable=True)
    receipt_image = Column(String(255), nullable=True)
    status = Column(String(30), default="pending")  # pending, approved, rejected
    created_at = Column(DateTime, default=datetime.utcnow)
    approved_at = Column(DateTime, nullable=True)
    admin_note = Column(Text, nullable=True)


class DiagnosticTest(Base):
    """O'qituvchi tomonidan fan bo'yicha yaratilgan diagnostika testi (audio/yozma hint bilan)."""
    __tablename__ = "diagnostic_tests"
    id = Column(String(36), primary_key=True, default=gen_uuid)
    teacher_id = Column(String(36), ForeignKey("users.id"), index=True, nullable=False)
    subject_id = Column(String(36), ForeignKey("teacher_subjects.id"), index=True, nullable=False)
    title = Column(String(255), nullable=False)
    description = Column(Text, nullable=True)
    hint_type = Column(String(20), default="written")  # "audio", "written", "mixed"
    time_limit_minutes = Column(Integer, default=30)
    max_attempts = Column(Integer, default=1)
    is_active = Column(Boolean, default=True)
    is_published = Column(Boolean, default=False)
    cefr_target = Column(String(10), default="B1")  # Maqsadlangan CEFR darajasi
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

    questions = relationship("DiagnosticQuestion", back_populates="test", cascade="all, delete-orphan")
    attempts = relationship("DiagnosticAttempt", back_populates="test")
```

```python


class DiagnosticQuestion(Base):
    """Diagnostika test savoli (audio yoki yozma hint bilan)."""
    __tablename__ = "diagnostic_questions"
    id = Column(String(36), primary_key=True, default=gen_uuid)
    test_id = Column(String(36), ForeignKey("diagnostic_tests.id"), index=True, nullable=False)
    question_order = Column(Integer, default=1)
    skill = Column(String(20), nullable=False)  # listening, reading, writing, speaking, grammar, vocabulary
    cefr_level = Column(String(2), default="B1")
    question_text = Column(Text, nullable=False)  # Asosiy savol matni
    question_type = Column(String(20), default="multiple_choice")  # multiple_choice, true_false, fill_blank, audio_response

    # Variantlar (JSON formatida)
    options = Column(JSON, nullable=True)  # [{"text": "A", "is_correct": true}, ...]
    correct_answer = Column(Text, nullable=True)  # To'g'ri javob matni (fill_blank uchun)

    # Hint (dest) tizimi
    hint_type = Column(String(20), default="written")  # "audio", "written", "none"
    hint_text = Column(Text, nullable=True)  # Yozma hint matni
    hint_audio_url = Column(String(500), nullable=True)  # Audio hint URL
    hint_audio_text = Column(Text, nullable=True)  # Audio matni (transkriptsiya)

    # Tushuntirish
    explanation = Column(Text, nullable=True)
    points = Column(Integer, default=1)
    created_at = Column(DateTime, default=datetime.utcnow)

    test = relationship("DiagnosticTest", back_populates="questions")
    answers = relationship("DiagnosticAnswer", back_populates="question", cascade="all, delete-orphan")


class DiagnosticAttempt(Base):
    """Talabaning diagnostika testi urinish natijasi (real vaqtda)."""
    __tablename__ = "diagnostic_attempts"
    id = Column(String(36), primary_key=True, default=gen_uuid)
    test_id = Column(String(36), ForeignKey("diagnostic_tests.id"), index=True, nullable=False)
    student_id = Column(String(36), ForeignKey("users.id"), index=True, nullable=False)
    attempt_number = Column(Integer, default=1)
    status = Column(String(20), default="in_progress")  # in_progress, completed, timed_out
    started_at = Column(DateTime, default=datetime.utcnow)
    completed_at = Column(DateTime, nullable=True)
    time_spent_seconds = Column(Integer, default=0)

    # Natijalar
    total_score = Column(Integer, default=0)
    max_score = Column(Integer, default=0)
    percentage = Column(Integer, default=0)
    grade = Column(String(20), nullable=True)  # A, B, C, D, F yoki 5,4,3,2

    # Ko'nikma bo'yicha natijalar (JSON)
    skill_breakdown = Column(JSON, nullable=True)  # {"listening": {"correct": 3, "total": 5, "percentage": 60}, ...}
    cefr_estimate = Column(JSON, nullable=True)  # {"listening": "B1", "reading": "B2", ...}

    # Xatolik tahlili
    wrong_answers = Column(JSON, nullable=True)  # [{question_id, skill, student_answer, correct_answer, hint_used}, ...]
    hints_used = Column(JSON, nullable=True)  # {question_id: "audio"|"written", ...}

    test = relationship("DiagnosticTest", back_populates="attempts")
    answers = relationship("DiagnosticAnswer", back_populates="attempt", cascade="all, delete-orphan")


class DiagnosticAnswer(Base):
    """Talabaning bitta savolga bergan javobi."""
    __tablename__ = "diagnostic_answers"
    id = Column(String(36), primary_key=True, default=gen_uuid)
    attempt_id = Column(String(36), ForeignKey("diagnostic_attempts.id"), index=True, nullable=False)
    question_id = Column(String(36), ForeignKey("diagnostic_questions.id"), index=True, nullable=False)
    selected_option_index = Column(Integer, nullable=True)  # Multiple choice uchun
    answer_text = Column(Text, nullable=True)  # Fill-in-the-blank, audio response uchun
    is_correct = Column(Boolean, default=False)
    hint_used = Column(String(20), nullable=True)  # "audio", "written", "none"
    time_spent_seconds = Column(Integer, default=0)
    answered_at = Column(DateTime, default=datetime.utcnow)

    attempt = relationship("DiagnosticAttempt", back_populates="answers")
    question = relationship("DiagnosticQuestion", back_populates="answers")


class SubjectBook(Base):
    """Fanga bog'liq kitoblar (asosiy va qo'shimcha adabiyot)."""
    __tablename__ = "subject_books"
    id = Column(String(36), primary_key=True, default=gen_uuid)
    subject_id = Column(String(36), ForeignKey("teacher_subjects.id"), index=True, nullable=False)
    teacher_id = Column(String(36), ForeignKey("users.id"), index=True, nullable=False)
    title = Column(String(255), nullable=False)
    author = Column(String(255), nullable=True)
    isbn = Column(String(50), nullable=True)
    publisher = Column(String(255), nullable=True)
    year = Column(SmallInteger, nullable=True)
    edition = Column(String(50), nullable=True)
```

```
    book_type = Column(String(20), default="main")  # "main", "supplementary", "reference"
    description = Column(Text, nullable=True)
    cover_image_url = Column(String(500), nullable=True)
    file_url = Column(String(500), nullable=True)  # PDF/EPUB fayl yo'li
    content_text = Column(Text, nullable=True)  # Matn ko'rinishida (AI uchun)
    is_active = Column(Boolean, default=True)
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)


class CustomDiagnosticQuestion(Base):
    """O'qituvchi tomonidan yuklangan kirish diagnostika savoli (talaba ekranidagi JSON shakli bilan bir xil)."""
    __tablename__ = "custom_diagnostic_questions"
    id = Column(String(36), primary_key=True, default=gen_uuid)
    teacher_id = Column(String(36), ForeignKey("users.id"), index=True, nullable=False)
    subject_id = Column(String(36), ForeignKey("teacher_subjects.id"), index=True, nullable=True)  # NULL = umumiy (barcha fanlar)
    question_order = Column(Integer, default=1)
    skill = Column(String(20), default="reading")  # listening, reading, writing, speaking
    cefr_level = Column(String(2), default="B1")
    instruction = Column(Text, nullable=True)
    question = Column(Text, nullable=False)
    options = Column(JSON, nullable=False)  # ["A matni", "B matni", ...]
    correct_index = Column(Integer, default=0)
    rationale = Column(Text, nullable=True)
    is_audio = Column(Boolean, default=False)
    audio_url = Column(String(500), nullable=True)  # /uploads/audio/<fayl>
    audio_filename = Column(String(255), nullable=True)  # asl fayl nomi
    audio_text = Column(Text, nullable=True)  # transkripsiya (audio ochilmasa TTS zaxirasi)
    is_active = Column(Boolean, default=True)
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)


class CustomSurveyQuestion(Base):
    """O'qituvchi tomonidan yuklangan kognitiv-affektiv anketa (FLCAS) savoli."""
    __tablename__ = "custom_survey_questions"
    id = Column(String(36), primary_key=True, default=gen_uuid)
    teacher_id = Column(String(36), ForeignKey("users.id"), index=True, nullable=False)
    subject_id = Column(String(36), ForeignKey("teacher_subjects.id"), index=True, nullable=True)  # NULL = umumiy (barcha fanlar)
    question_order = Column(Integer, default=1)
    dimension = Column(String(30), default="anxiety")  # anxiety, load_capacity, pace, autonomy, motivation
    prompt_uz = Column(Text, nullable=False)
    prompt_en = Column(Text, nullable=True)
    weight = Column(Float, default=1.0)
    is_active = Column(Boolean, default=True)
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)


class LessonRecord(Base):
    """
    Yakunlangan 5 bosqichli mashg'ulotning to'liq yozuvi (dissertatsiya 3.3: refleksiya moduli
    "o'zini baholash varaqasi va dinamika ko'rsatkichlari"; 4.1: "kuzatuv varaqasi" jarayon ko'rsatkichlari).
    Metakognitiv sikl: rejalashtirish (planning) -> monitoring -> baholash (evaluation) -> refleksiya.
    """
    __tablename__ = "lesson_records"
    id = Column(String(36), primary_key=True, default=gen_uuid)
    student_id = Column(String(36), ForeignKey("users.id"), index=True, nullable=False)
    group_id = Column(String(36), ForeignKey("groups.id"), nullable=True)
    skill = Column(String(20), default="writing")
    task_variant = Column(String(10), nullable=True)  # A (yuqori) / B (o'rta) / C (quyi)
    scaffold_level = Column(String(20), nullable=True)
    # Rejalashtirish (1-bosqich)
    plan_goal = Column(Text, nullable=True)
    plan_strategy = Column(Text, nullable=True)
    # Monitoring (3-bosqich jarayon ko'rsatkichlari)
    help_requests = Column(Integer, default=0)        # qulflangan AI tugmasini bosish urinishlari
    self_checks = Column(Integer, default=0)          # o'z-o'zini tekshirish belgilari
    independent_seconds = Column(Integer, default=0)  # mustaqil bosqichda sarflangan vaqt
    total_seconds = Column(Integer, default=0)
    word_count = Column(Integer, default=0)
    production_text = Column(Text, nullable=True)
    # Baholash (4–5-bosqich)
    score = Column(SmallInteger, nullable=True)
    rubric = Column(JSON, nullable=True)
    self_rating = Column(JSON, nullable=True)
    rating_gap = Column(Float, nullable=True)         # |o'z bahosi - tizim bahosi| o'rtacha (metakognitiv aniqlik)
    feedback_decisions = Column(JSON, nullable=True)  # [{id, title, accepts, justification}]
    l1_pattern_ids = Column(JSON, nullable=True)      # takrorlanuvchi xatolarni kuzatish uchun
    comparison_note = Column(Text, nullable=True)     # qiyosiy usul: o'z varianti va AI varianti farqi asosi
    ai_powered = Column(Boolean, default=False)
    # Refleksiya
    gap_explanation = Column(Text, nullable=True)
    next_goal = Column(Text, nullable=True)
    teacher_extension = Column(Text, nullable=True)   # o'qituvchi izohni jarayon/o'zini boshqarish darajasiga ko'taradi
    teacher_id = Column(String(36), nullable=True)
    created_at = Column(DateTime, default=datetime.utcnow, index=True)


```

```
class TaskVariantAssignment(Base):
    """Tabaqalashtirish: o'qituvchi talabaga topshiriq variantini (A/B/C) biriktiradi yoki tizim taklifini o'zgartiradi."""
    __tablename__ = "task_variant_assignments"
    id = Column(String(36), primary_key=True, default=gen_uuid)
    student_id = Column(String(36), ForeignKey("users.id"), index=True, nullable=False)
    teacher_id = Column(String(36), ForeignKey("users.id"), nullable=True)
    variant = Column(String(10), nullable=False)  # A / B / C
    note = Column(Text, nullable=True)
    source = Column(String(20), default="teacher")  # teacher | system
    created_at = Column(DateTime, default=datetime.utcnow)


class ApiKey(Base):
    """
    Dasturchi (developer) va Super Admin (admin) uchun dasturiy integratsiya API kaliti.
    X-API-Key: tilora_sk_live_... yoki Authorization: ApiKey ... orqali autentifikatsiya qilinadi.
    """
    __tablename__ = "api_keys"
    id = Column(String(36), primary_key=True, default=gen_uuid)
    user_id = Column(String(36), ForeignKey("users.id"), index=True, nullable=False)
    name = Column(String(100), nullable=False)  # masalan: "Mobil ilova integratsiyasi"
    key_prefix = Column(String(30), nullable=False)  # masalan: "tilora_sk_••••7890"
    key_hash = Column(String(64), unique=True, index=True, nullable=False)  # SHA-256 xesh
    role = Column(String(20), default="developer")  # developer, admin
    is_active = Column(Boolean, default=True)
    rate_limit = Column(Integer, default=120)  # daqiqasiga ruxsat etilgan so'rovlar soni
    last_used_at = Column(DateTime, nullable=True)
    expires_at = Column(DateTime, nullable=True)
    created_at = Column(DateTime, default=datetime.utcnow)
```

Fayl: backend/app/api/auth.py

Auth router: ro'yxatdan o'tish, kirish, profil, parolni o'zgartirish

```
import re
from datetime import datetime, timezone
from typing import Dict, Any, Optional
from fastapi import APIRouter, Depends, HTTPException, Request, status
from sqlalchemy.orm import Session
from app.config import settings
from app.database import get_db
from app.models.entities import User, Institution, Group, StudentProfile, SubscriptionPayment
from app.schemas.all_schemas import LoginRequest, RegisterRequest, SubscribeRequest, UpdateCredentialsRequest
from app.core.security import hash_password, verify_password, create_access_token, needs_rehash
from app.core.deps import get_current_user
from app.core.ratelimit import SlidingWindowLimiter, client_ip

router = APIRouter(prefix="/auth", tags=["Autentifikatsiya"])

# Brute-force himoyasi: IP va login bo'yicha alohida hisoblanadi
_login_ip_limiter = SlidingWindowLimiter(settings.LOGIN_MAX_ATTEMPTS * 3, settings.LOGIN_WINDOW_SECONDS)
_login_ident_limiter = SlidingWindowLimiter(settings.LOGIN_MAX_ATTEMPTS, settings.LOGIN_WINDOW_SECONDS)
_register_limiter = SlidingWindowLimiter(20, 3600)

_INVALID_CREDENTIALS = "Login (email/telefon) yoki parol noto'g'ri."
_EMAIL_RE = re.compile(r"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$")
_LOGIN_RE = re.compile(r"^[A-Za-z0-9._-]{3,64}$")


def _utcnow() -> datetime:
    return datetime.now(timezone.utc).replace(tzinfo=None)


def validate_password_strength(password: str) -> None:
    """Parol siyosati: kamida PASSWORD_MIN_LENGTH belgi, harf va raqam ishtiroki."""
    pwd = password or ""
    if len(pwd) < settings.PASSWORD_MIN_LENGTH:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail=f"Parol kamida {settings.PASSWORD_MIN_LENGTH} ta belgidan iborat bo'lishi kerak."
        )
    if len(pwd) > 128:
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail="Parol 128 belgidan oshmasligi kerak.")
    if not re.search(r"[A-Za-z]", pwd) or not re.search(r"\d", pwd):
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Parolda kamida bitta harf va bitta raqam bo'lishi kerak (masalan: Talaba2026)."
        )
    weak = {"12345678", "password", "password1", "qwerty123", "<REDACTED>", "<REDACTED>"}
    if pwd.lower() in weak:
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail="Bu parol juda keng tarqalgan. Boshqa parol tanlang.")


def validate_login_identifier(ident: str) -> str:
    """Login yoki email formatini tekshiradi va kichik harflarga keltiradi."""
    ident = (ident or "").strip().lower()
    if not ident:
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail="Login yoki email kiritilishi shart.")
```

```
    if "@" in ident:
        if not _EMAIL_RE.match(ident):
            raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail="Email manzili noto'g'ri formatda.")
    elif not _LOGIN_RE.match(ident):
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Login 3–64 ta belgidan iborat bo'lib, faqat lotin harflari, raqam, nuqta, chiziqcha va pastki chiziqdan tashkil topishi kerak."
        )
    return ident


def _user_public(user: User, group_name: Optional[str]) -> Dict[str, Any]:
    return {
        "id": user.id,
        "email": user.email,
        "full_name": user.full_name,
        "role": user.role,
        "phone_number": user.phone_number,
        "group_id": user.group_id,
        "group_name": group_name,
        "institution_id": user.institution_id,
        "is_approved": bool(getattr(user, "is_approved", True)),
        "subscription_tier": getattr(user, "subscription_tier", "free") or "free",
        "subscription_status": getattr(user, "subscription_status", "active") or "active"
    }


def _group_name(db: Session, user: User) -> Optional[str]:
    if not user.group_id:
        return None
    grp = db.query(Group).filter(Group.id == user.group_id).first()
    return grp.name if grp else None


def _issue_token(user: User) -> str:
    return create_access_token({
        "id": user.id,
        "email": user.email,
        "role": user.role,
        "full_name": user.full_name,
        "institution_id": user.institution_id,
        "group_id": user.group_id,
        "subscription_tier": getattr(user, "subscription_tier", "free") or "free",
        "is_approved": bool(getattr(user, "is_approved", True))
    })


def _find_user_by_identifier(db: Session, raw_ident: str) -> Optional[User]:
    ident = raw_ident.strip().lower()
    user = db.query(User).filter(User.email == ident).first()
    if not user and "@" not in ident and _LOGIN_RE.match(ident):
        user = db.query(User).filter(User.email == f"{ident}@tilora.uz").first()
        if not user:
            # Masalan: "ubek51240" deb kiritilganda "ubek51240@gmail.com" ni topish
            user = db.query(User).filter(User.email.like(f"{ident}@%")).first()
    if not user:
        digits = re.sub(r"\D", "", raw_ident)
        if len(digits) >= 9:
            last9 = digits[-9:]
            user = db.query(User).filter(User.phone_number.like(f"%{last9}%")).first()
    return user


@router.post("/login")
def login(payload: LoginRequest, request: Request, db: Session = Depends(get_db)):
    raw_ident = (payload.email or "").strip()[:255]
    ip = client_ip(request)
    ident_key = f"login:{raw_ident.lower()}"
    ip_key = f"login_ip:{ip}"

    for limiter, key in ((_login_ip_limiter, ip_key), (_login_ident_limiter, ident_key)):
        limited, retry = limiter.is_limited(key)
        if limited:
            raise HTTPException(
                status_code=status.HTTP_429_TOO_MANY_REQUESTS,
                detail=f"Juda ko'p noto'g'ri urinish. {max(1, retry // 60)} daqiqadan so'ng qayta urinib ko'ring.",
                headers={"Retry-After": str(retry)},
            )

    user = _find_user_by_identifier(db, raw_ident) if raw_ident else None

    # Foydalanuvchi mavjudligi va parol xatosi bir xil xabar bilan qaytariladi (enumeration himoyasi)
    if not user or not verify_password(payload.password or "", user.password_hash):
        _login_ip_limiter.record(ip_key)
        _login_ident_limiter.record(ident_key)
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail=_INVALID_CREDENTIALS)

    # Super Admin tasdiqlaganlik tekshiruvi (Admin tasdiqlamasa kirish taqiqlanadi)
    is_approved = bool(getattr(user, "is_approved", True))
    if not is_approved and user.role != "admin":
```

```
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="Sizning hisobingiz hali Super Administrator tomonidan tasdiqlanmagan. Iltimos, admin arizangizni tasdiqlash
    ini kuting!"
        )

    is_active = bool(getattr(user, "is_active", True))
    if not is_active and user.role != "admin":
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="Sizning hisobingiz nofaol holatda. Administratorga murojaat qiling."
        )

    # Eski (statik tuzli) xeshni yangi, foydalanuvchiga xos tuzli formatga ko'chirish
    if needs_rehash(user.password_hash):
        user.password_hash = hash_password(payload.password)
        db.commit()

    _login_ident_limiter.reset(ident_key)

    return {
        "access_token": _issue_token(user),
        "token_type": "bearer",
        "expires_in": settings.ACCESS_TOKEN_EXPIRE_MINUTES * 60,
        "user": _user_public(user, _group_name(db, user))
    }


@router.post("/register")
def register(payload: RegisterRequest, request: Request, db: Session = Depends(get_db)):
    _register_limiter.check_and_record(
        f"register:{client_ip(request)}",
        "Ro'yxatdan o'tish urinishlari ko'payib ketdi. Biroz vaqtdan so'ng qayta urinib ko'ring."
    )

    raw_ident = (payload.email or "").strip()
    email_clean = validate_login_identifier(raw_ident)
    validate_password_strength(payload.password)

    full_name = (payload.full_name or "").strip()
    if len(full_name) < 3 or len(full_name) > 120:
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail="Ism-sharif 3–120 belgi oralig'ida bo'lishi kerak."
    )

    # 1. Email yoki login bo'yicha mavjudligini tekshirish
    existing = db.query(User).filter(User.email == email_clean).first()
    if not existing and "@" not in email_clean:
        existing = db.query(User).filter(
            (User.email == f"{email_clean}@tilora.uz") | (User.email.like(f"{email_clean}@%"))
        ).first()

    if existing:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail=f"«{raw_ident}» logini / emaili bilan foydalanuvchi allaqachon ro'yxatdan o'tgan."
        )

    # 2. Agar telefon kiritilgan bo'lsa, telefon bo'yicha ham tekshirish
    phone_clean = payload.phone_number.strip()[:30] if payload.phone_number else None
    if phone_clean:
        phone_digits = re.sub(r"\D", "", phone_clean)
        if len(phone_digits) < 9 or len(phone_digits) > 15:
            raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail="Telefon raqami noto'g'ri formatda (masalan: +9
    98901234567).")
        last9 = phone_digits[-9:]
        existing_phone = db.query(User).filter(User.phone_number.like(f"%{last9}%")).first()
        if existing_phone:
            raise HTTPException(
                status_code=status.HTTP_400_BAD_REQUEST,
                detail=f"«{phone_clean}» telefon raqami allaqachon boshqa foydalanuvchiga biriktirilgan."
            )

    # Talabalar login sahifasida registratsiya qiladi (o'qituvchilar esa faqat Super Admin tomonidan yaratiladi).
    # payload.role e'tiborga olinmaydi — rolni foydalanuvchi o'zi tanlay olmaydi.
    role = "student"

    # Muassasa: faqat mavjud OTM qabul qilinadi, aks holda standart
    institution_id = payload.institution_id or "inst_buxdu"
    if not db.query(Institution).filter(Institution.id == institution_id).first():
        institution_id = "inst_buxdu" if db.query(Institution).filter(Institution.id == "inst_buxdu").first() else None

    # Guruhni aniqlash yoki yaratish
    group_id = None
    if payload.group_id and db.query(Group).filter(Group.id == payload.group_id).first():
        group_id = payload.group_id
    elif payload.group_name and payload.group_name.strip():
        g_name = payload.group_name.strip()[:50]
        grp = db.query(Group).filter(Group.name == g_name).first()
        if not grp:
            grp = Group(name=g_name, institution_id=institution_id, course_year=3, experiment_arm="experimental")
            db.add(grp)
```

```
            db.commit()
            db.refresh(grp)
        group_id = grp.id

    new_user = User(
        email=email_clean,
        password_hash=hash_password(payload.password),
        full_name=full_name,
        role=role,
        institution_id=institution_id,
        group_id=group_id,
        is_approved=False,  # Super admin tasdiqlashi shart!
        subscription_tier="free",
        subscription_status="active",
        phone_number=phone_clean
    )
    db.add(new_user)
    db.commit()
    db.refresh(new_user)

    # Talaba boshlang'ich profili
    prof = StudentProfile(
        student_id=new_user.id,
        cefr_listening="B1",
        cefr_reading="B2",
        cefr_speaking="A2",
        cefr_writing="B1",
        load_capacity=50,
        pace_index=50,
        anxiety_level=60,
        autonomy_readiness=40,
        source="initial_registration"
    )
    db.add(prof)
    db.commit()

    return {
        "status": "pending_approval",
        "message": f"Hurmatli {new_user.full_name}, ro'yxatdan o'tish arizangiz qabul qilindi! Super administrator tasdiqlagani
    dan so'ng tizimga login va parolingiz bilan kira olasiz.",
        "user": {
            "id": new_user.id,
            "email": new_user.email,
            "full_name": new_user.full_name,
            "role": new_user.role,
            "is_approved": False,
            "subscription_tier": "free"
        }
    }


@router.post("/subscribe")
def create_subscription_request(
    payload: SubscribeRequest,
    db: Session = Depends(get_db),
    current_user: dict = Depends(get_current_user)
):
    """
    Foydalanuvchi PRO obuna uchun to'lov arizasi topshiradi (Click, Payme, Karta).
    Admin tasdiqlagach PRO tarif faollashadi.
    """
    user_id = current_user.get("id")
    user = db.query(User).filter(User.id == user_id).first()
    if not user:
        raise HTTPException(status_code=404, detail="Foydalanuvchi topilmadi.")

    # Bir foydalanuvchida ko'p sonli kutilayotgan arizalar yig'ilib qolmasin
    pending_count = db.query(SubscriptionPayment).filter(
        SubscriptionPayment.user_id == user.id, SubscriptionPayment.status == "pending"
    ).count()
    if pending_count >= 3:
        raise HTTPException(status_code=400, detail="Sizda allaqachon ko'rib chiqilayotgan to'lov arizalari bor. Admin tasdig'i
    ni kuting.")

    plan = (payload.plan or "pro")[:50]
    if plan not in ("pro", "pro_monthly", "pro_annual"):
        plan = "pro_monthly"
    payment_method = (payload.payment_method or "click")[:50]
    if payment_method not in ("click", "payme", "card", "uzum"):
        payment_method = "click"
    amount = 49000.0 if plan != "pro_annual" else 490000.0  # Summa serverda belgilanadi, mijoz o'zgartira olmaydi
    tx_raw = (payload.transaction_id or "").strip()[:100]
    tx_id = tx_raw or f"TX-{_utcnow().strftime('%Y%m%d%H%M%S')}"

    payment = SubscriptionPayment(
        user_id=user.id,
        user_name=user.full_name,
        user_email=user.email,
        role=user.role,
        plan=plan,
        amount=amount,
```

```
        payment_method=payment_method,
        transaction_id=tx_id,
        status="pending"
    )
    db.add(payment)
    user.subscription_status = "pending"
    db.commit()
    db.refresh(payment)

    return {
        "status": "pending",
        "message": "To'lov arizangiz qabul qilindi! Super Administrator to'lovni tekshirib tasdiqlagach, PRO profilingiz avtoma
    tik ishga tushadi.",
        "payment_id": payment.id,
        "amount": amount,
        "plan": plan
    }


@router.get("/subscription-status")
def get_subscription_status(
    db: Session = Depends(get_db),
    current_user: dict = Depends(get_current_user)
):
    """Joriy foydalanuvchining obuna holati (Free yoki Pro)."""
    user_id = current_user.get("id")
    user = db.query(User).filter(User.id == user_id).first()
    if not user:
        raise HTTPException(status_code=404, detail="Foydalanuvchi topilmadi.")

    recent_payments = db.query(SubscriptionPayment).filter(
        SubscriptionPayment.user_id == user_id
    ).order_by(SubscriptionPayment.created_at.desc()).limit(5).all()

    return {
        "user_id": user.id,
        "full_name": user.full_name,
        "role": user.role,
        "subscription_tier": getattr(user, "subscription_tier", "free") or "free",
        "subscription_status": getattr(user, "subscription_status", "active") or "active",
        "is_pro": (getattr(user, "subscription_tier", "free") == "pro"),
        "payments": [
            {
                "id": p.id,
                "plan": p.plan,
                "amount": p.amount,
                "payment_method": p.payment_method,
                "status": p.status,
                "created_at": p.created_at.strftime("%Y-%m-%d %H:%M") if p.created_at else ""
            }
            for p in recent_payments
        ]
    }


@router.get("/me")
def get_me(db: Session = Depends(get_db), current_user: dict = Depends(get_current_user)):
    user_id = current_user.get("id")
    user = db.query(User).filter(User.id == user_id).first()
    if not user:
        return current_user
    return _user_public(user, _group_name(db, user))


@router.put("/change-credentials")
@router.post("/change-credentials")
def change_credentials(
    payload: UpdateCredentialsRequest,
    db: Session = Depends(get_db),
    current_user: dict = Depends(get_current_user)
):
    """
    Foydalanuvchi o'z loginini va/yoki parolini o'zgartiradi.
    Xavfsizlik uchun amaldagi parol (current_password) tekshiriladi.
    """
    user_id = current_user.get("id")
    user = db.query(User).filter(User.id == user_id).first()
    if not user:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Foydalanuvchi topilmadi.")

    # 1. Amaldagi parolni tekshirish (majburiy)
    if not payload.current_password or not verify_password(payload.current_password, user.password_hash):
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Amaldagi parolingiz noto'g'ri kiritildi! Iltimos, joriy parolingizni tekshirib qayta kiriting."
        )

    changes_made = []

    # 2. Yangi login tekshiruvi va o'zgartirish
    new_login = (payload.new_login or "").strip().lower()
```

```
    if new_login and new_login != user.email.lower():
        new_login = validate_login_identifier(new_login)
        existing = db.query(User).filter(
            User.id != user.id,
            (User.email == new_login) |
            (User.email == f"{new_login}@tilora.uz") |
            (User.email.like(f"{new_login}@%"))
        ).first()
        if existing:
            raise HTTPException(
                status_code=status.HTTP_400_BAD_REQUEST,
                detail=f"«{payload.new_login.strip()}» logini allaqachon boshqa foydalanuvchi tomonidan band qilingan. Boshqa l
ogin tanlang."
            )
        user.email = new_login
        changes_made.append("login")

    # 3. Yangi parol tekshiruvi va o'zgartirish
    new_pass = (payload.new_password or "").strip()
    if new_pass:
        validate_password_strength(new_pass)
        if verify_password(new_pass, user.password_hash):
            raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail="Yangi parol joriy paroldan farq qilishi kerak.
")
        user.password_hash = hash_password(new_pass)
        changes_made.append("parol")

    # 4. Ism-sharif (agar yuborilgan bo'lsa)
    if payload.full_name is not None and payload.full_name.strip() and payload.full_name.strip() != user.full_name:
        user.full_name = payload.full_name.strip()[:120]
        changes_made.append("ism-sharif")

    # 5. Telefon raqam (agar yuborilgan bo'lsa)
    if payload.phone_number is not None and payload.phone_number.strip() != (user.phone_number or ""):
        user.phone_number = payload.phone_number.strip()[:30]
        changes_made.append("telefon raqami")

    if not changes_made:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Hech qanday o'zgarish kiritilmadi. Yangi login yoki yangi parol kiriting."
        )

    db.commit()
    db.refresh(user)

    summary = ", ".join(changes_made)
    return {
        "status": "success",
        "message": f"Sizning {summary} ma'lumotlaringiz muvaffaqiyatli o'zgartirildi!",
        "access_token": _issue_token(user),
        "token_type": "bearer",
        "user": _user_public(user, _group_name(db, user))
    }


@router.post("/logout")
def logout():
    """Foydalanuvchi tizimdan chiqishi (sessiyani yakunlash). Token mijoz tomonida o'chiriladi."""
    return {"message": "Tizimdan muvaffaqiyatli chiqildi", "status": "logged_out"}
```

Fayl: backend/app/api/admin.py

Admin router: foydalanuvchilarni tasdiqlash, tarif, parolni tiklash

```
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session
from typing import Optional, Dict, Any, List
from datetime import datetime

from app.database import get_db
from app.models.entities import User, SubscriptionPayment, Institution, Group, TeacherSubject
from app.core.deps import require_role, get_current_user
from app.core.security import hash_password
from app.schemas.all_schemas import CreateTeacherRequest
from app.api.auth import validate_password_strength, validate_login_identifier

router = APIRouter(prefix="/admin", tags=["Administrator"], dependencies=[Depends(require_role(["admin"]))])

@router.get("/summary")
def get_admin_summary(db: Session = Depends(get_db)):
    total_users = db.query(User).count()
    pending_users = db.query(User).filter(User.is_approved == False).count()
    students_count = db.query(User).filter(User.role == "student").count()
    teachers_count = db.query(User).filter(User.role == "teacher").count()
    pro_users_count = db.query(User).filter(User.subscription_tier == "pro").count()
    pending_payments = db.query(SubscriptionPayment).filter(SubscriptionPayment.status == "pending").count()

    return {
        "total_users": total_users,
```

```
        "pending_users_count": pending_users,
        "students_count": students_count,
        "teachers_count": teachers_count,
        "pro_users_count": pro_users_count,
        "pending_payments_count": pending_payments,
        "institutions": ["Buxoro davlat universiteti", "Nukus DPI", "SamDChTI"],
        "system_status": "healthy"
    }

# ============================================================
# 1. FOYDALANUVCHILARNI TASDIQLASH (APPROVAL QUEUE)
# ============================================================

@router.get("/pending-users")
def get_pending_users(db: Session = Depends(get_db)):
    """
    Ro'yxatdan o'tgan va Super Admin tasdiqlashini kutayotgan foydalanuvchilar.
    """
    users = db.query(User).filter(User.is_approved == False).order_by(User.created_at.desc()).all()
    result = []
    for u in users:
        result.append({
            "id": u.id,
            "full_name": u.full_name,
            "email": u.email,
            "role": u.role,
            "phone_number": getattr(u, "phone_number", None) or "Mavjud emas",
            "subscription_tier": getattr(u, "subscription_tier", "free") or "free",
            "created_at": u.created_at.strftime("%Y-%m-%d %H:%M") if u.created_at else ""
        })
    return {"pending_users": result, "total": len(result)}


@router.post("/users/{user_id}/approve")
def approve_user_registration(user_id: str, db: Session = Depends(get_db)):
    """
    Super Admin yangi ro'yxatdan o'tgan talaba yoki o'qituvchini tasdiqlaydi.
    Shundan so'ng u login va paroli bilan kira oladi.
    """
    user = db.query(User).filter(User.id == user_id).first()
    if not user:
        raise HTTPException(status_code=404, detail="Foydalanuvchi topilmadi.")

    user.is_approved = True
    user.is_active = True
    db.commit()

    return {
        "status": "approved",
        "message": f"{user.full_name} ({user.role}) arizasi Super Admin tomonidan tasdiqlandi. Endi u tizimga kira oladi."
    }


@router.post("/users/{user_id}/reject")
def reject_user_registration(user_id: str, db: Session = Depends(get_db)):
    """
    Super Admin ro'yxatdan o'tgan nomaqbul hisobni rad etadi va o'chiradi.
    """
    user = db.query(User).filter(User.id == user_id).first()
    if not user:
        raise HTTPException(status_code=404, detail="Foydalanuvchi topilmadi.")
    if user.role == "admin":
        raise HTTPException(status_code=400, detail="Administrator hisobini rad etib bo'lmaydi.")

    full_name = user.full_name
    db.delete(user)
    db.commit()

    return {
        "status": "rejected",
        "message": f"{full_name} arizasi bekor qilindi va tizimdan o'chirildi."
    }


@router.get("/users")
def get_all_users(role: Optional[str] = None, db: Session = Depends(get_db)):
    """
    Barcha ro'yxatdagi foydalanuvchilar (roli, tasdiq holati va obuna darajasi bilan).
    """
    query = db.query(User)
    if role:
        query = query.filter(User.role == role)
    users = query.order_by(User.created_at.desc()).all()

    result = []
    for u in users:
        result.append({
            "id": u.id,
            "full_name": u.full_name,
            "email": u.email,
            "role": u.role,
```

```
            "is_approved": getattr(u, "is_approved", True),
            "subscription_tier": getattr(u, "subscription_tier", "free") or "free",
            "created_at": u.created_at.strftime("%Y-%m-%d %H:%M") if u.created_at else ""
        })
    return {"users": result, "total": len(result)}


@router.post("/users/{user_id}/set-tier")
def set_user_subscription_tier(
    user_id: str,
    payload: Dict[str, Any],
    db: Session = Depends(get_db)
):
    """
    Admin tomonidan foydalanuvchini bevosita PRO yoki Free tarifiga o'tkazish.
    """
    user = db.query(User).filter(User.id == user_id).first()
    if not user:
        raise HTTPException(status_code=404, detail="Foydalanuvchi topilmadi.")

    tier = payload.get("subscription_tier", "pro")
    if tier not in ["free", "pro"]:
        raise HTTPException(status_code=400, detail="Noto'g'ri tarif. Faqat 'free' yoki 'pro' mumkin.")

    user.subscription_tier = tier
    user.subscription_status = "active"
    db.commit()

    return {
        "status": "success",
        "message": f"{user.full_name} ning obunasi '{tier.upper()}' tarifiga o'tkazildi."
    }


@router.post("/users/{user_id}/reset-password")
def reset_user_password(
    user_id: str,
    payload: Dict[str, Any],
    db: Session = Depends(get_db)
):
    """
    Super Admin istalgan foydalanuvchining (talaba yoki o'qituvchi) parolini yangilaydi.
    """
    user = db.query(User).filter(User.id == user_id).first()
    if not user:
        raise HTTPException(status_code=404, detail="Foydalanuvchi topilmadi.")

    new_pwd = str(payload.get("password") or "").strip()
    validate_password_strength(new_pwd)

    user.password_hash = hash_password(new_pwd)
    # Tasdiqlanmagan bo'lsa avtomatik tasdiqlash
    user.is_approved = True
    user.is_active = True
    db.commit()

    # Yangi parol javobda qaytarilmaydi (loglar va brauzer tarixida qolmasligi uchun)
    return {
        "status": "success",
        "message": f"{user.full_name} ning paroli muvaffaqiyatli yangilandi.",
        "user_email": user.email
    }


# ============================================================
# 2. TO'LOVLAR VA PRO OBUNALAR (PAYMENTS & PRO UPGRADE)
# ============================================================

@router.get("/payments")
def get_subscription_payments(
    status_filter: Optional[str] = None,
    db: Session = Depends(get_db)
):
    """
    Barcha to'lov arizalari (Super Admin tekshirishi va PRO faollashtirishi uchun).
    """
    query = db.query(SubscriptionPayment)
    if status_filter:
        query = query.filter(SubscriptionPayment.status == status_filter)
    payments = query.order_by(SubscriptionPayment.created_at.desc()).all()

    result = []
    for p in payments:
        result.append({
            "id": p.id,
            "user_id": p.user_id,
            "user_name": p.user_name,
            "user_email": p.user_email,
            "role": p.role,
            "plan": p.plan,
            "amount": p.amount,
```

```
            "payment_method": p.payment_method,
            "transaction_id": p.transaction_id,
            "status": p.status,
            "created_at": p.created_at.strftime("%Y-%m-%d %H:%M") if p.created_at else "",
            "approved_at": p.approved_at.strftime("%Y-%m-%d %H:%M") if p.approved_at else ""
        })
    return {"payments": result, "total": len(result)}


@router.post("/payments/{payment_id}/approve")
def approve_subscription_payment(
    payment_id: str,
    db: Session = Depends(get_db)
):
    """
    Super Admin to'lovni tasdiqlaydi:
    To'lov holati 'approved' bo'ladi va foydalanuvchi 'pro' darajasiga o'tadi!
    """
    payment = db.query(SubscriptionPayment).filter(SubscriptionPayment.id == payment_id).first()
    if not payment:
        raise HTTPException(status_code=404, detail="To'lov arizasi topilmadi.")

    payment.status = "approved"
    payment.approved_at = datetime.utcnow()

    # Foydalanuvchini PRO tarifga o'tkazish
    user = db.query(User).filter(User.id == payment.user_id).first()
    if user:
        user.subscription_tier = "pro"
        user.subscription_status = "active"

    db.commit()

    return {
        "status": "approved",
        "message": f"{payment.user_name} ning {payment.amount:,} so'mlik to'lovi tasdiqlandi va uning hisobi '  PRO' tarifiga o
    'tkazildi!"
    }


@router.post("/payments/{payment_id}/reject")
def reject_subscription_payment(
    payment_id: str,
    payload: Optional[Dict[str, Any]] = None,
    db: Session = Depends(get_db)
):
    """
    Super Admin soxta yoki yetib kelmagan to'lovni rad etadi.
    """
    payment = db.query(SubscriptionPayment).filter(SubscriptionPayment.id == payment_id).first()
    if not payment:
        raise HTTPException(status_code=404, detail="To'lov arizasi topilmadi.")

    payment.status = "rejected"
    if payload and "admin_note" in payload:
        payment.admin_note = payload["admin_note"]

    user = db.query(User).filter(User.id == payment.user_id).first()
    if user and user.subscription_tier != "pro":
        user.subscription_status = "active"

    db.commit()

    return {
        "status": "rejected",
        "message": f"{payment.user_name} to'lov arizasi rad etildi."
    }


# ============================================================
# 3. O'QITUVCHILARNI BOSHQARISH VA YARATISH (TEACHER MANAGEMENT)
# ============================================================

@router.get("/teachers")
def get_all_teachers(db: Session = Depends(get_db)):
    """
    Super Admin uchun barcha o'qituvchilar ro'yxati va ularning fanlari.
    """
    teachers = db.query(User).filter(User.role == "teacher").order_by(User.created_at.desc()).all()
    result = []
    for t in teachers:
        subjects = db.query(TeacherSubject).filter(TeacherSubject.teacher_id == t.id).all()
        subj_list = [{"id": s.id, "name": s.name, "semester": s.semester} for s in subjects]
        result.append({
            "id": t.id,
            "full_name": t.full_name,
            "email": t.email,
            "phone_number": t.phone_number or "Ko'rsatilmagan",
            "subscription_tier": getattr(t, "subscription_tier", "pro") or "pro",
            "is_approved": getattr(t, "is_approved", True),
            "subjects": subj_list,
```

```
            "subjects_count": len(subj_list),
            "created_at": t.created_at.strftime("%Y-%m-%d %H:%M") if t.created_at else ""
        })
    return {"teachers": result, "total": len(result)}


@router.post("/teachers")
def create_teacher(payload: CreateTeacherRequest, db: Session = Depends(get_db)):
    """
    Super Admin yangi o'qituvchi yaratadi.
    Yangi o'qituvchi avtomatik tasdiqlangan (is_approved=True) va 'pro' tarif bilan faollashadi.
    """
    email_clean = validate_login_identifier(payload.email)
    validate_password_strength(payload.password.strip())
    if payload.subscription_tier not in (None, "free", "pro"):
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail="Tarif faqat 'free' yoki 'pro' bo'lishi mumkin.")

    # Email yoki login band emasligini tekshirish
    existing = db.query(User).filter(User.email == email_clean).first()
    if not existing and "@" not in email_clean:
        existing = db.query(User).filter(User.email == f"{email_clean}@tilora.uz").first()

    if existing:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail=f"'{payload.email}' login yoki emaili bilan foydalanuvchi allaqachon mavjud!"
        )

    # Yangi o'qituvchini yaratish
    new_teacher = User(
        email=email_clean,
        password_hash=hash_password(payload.password.strip()),
        full_name=payload.full_name.strip(),
        role="teacher",
        institution_id=payload.institution_id or "inst_buxdu",
        is_approved=True,  # Super admin yaratgani uchun to'g'ridan-to'g'ri faol
        is_active=True,
        subscription_tier=payload.subscription_tier or "pro",
        subscription_status="active",
        phone_number=payload.phone_number.strip() if payload.phone_number else None
    )
    db.add(new_teacher)
    db.commit()
    db.refresh(new_teacher)

    # Agar biriktirilgan fan nomi ko'rsatilgan bo'lsa, TeacherSubject jadvaliga ham qo'shamiz
    subject_created = None
    if payload.subject_name and payload.subject_name.strip():
        subj_name = payload.subject_name.strip()
        new_subj = TeacherSubject(
            teacher_id=new_teacher.id,
            name=subj_name,
            code="ENG-101",
            semester="5-semestr"
        )
        db.add(new_subj)
        db.commit()
        db.refresh(new_subj)
        subject_created = {"id": new_subj.id, "name": new_subj.name}

    return {
        "status": "success",
        "message": f"O'qituvchi {new_teacher.full_name} muvaffaqiyatli yaratildi va hisobi faollashtirildi.",
        "teacher": {
            "id": new_teacher.id,
            "full_name": new_teacher.full_name,
            "email": new_teacher.email,
            "role": "teacher",
            "phone_number": new_teacher.phone_number,
            "subscription_tier": new_teacher.subscription_tier,
            "is_approved": True,
            "subject": subject_created,
            "created_at": new_teacher.created_at.strftime("%Y-%m-%d %H:%M") if new_teacher.created_at else ""
        }
    }


@router.delete("/teachers/{teacher_id}")
def delete_teacher(teacher_id: str, db: Session = Depends(get_db)):
    """
    Super Admin o'qituvchi hisobini o'chiradi.
    """
    teacher = db.query(User).filter(User.id == teacher_id, User.role == "teacher").first()
    if not teacher:
        raise HTTPException(status_code=404, detail="O'qituvchi topilmadi.")

    full_name = teacher.full_name
    db.delete(teacher)
    db.commit()

    return {
```

```
        "status": "success",
        "message": f"O'qituvchi {full_name} hisobi muvaffaqiyatli o'chirildi."
    }
```